
Machine Learning Final Project Report

Sude Zarei

Student ID: 401130393

SudeZarei@gmail.com

1. Introduction

Quality of wine is an important factor for one is shopping for a wine. Based on Cortez et. al. investigations¹, wine industry is investing a lot of money in quality assessment and wine certification to safeguard human health and improve wine marketing. Wine certification is generally assessed by physicochemical and sensory tests². Taste of wine is a difficult factor to classify wine because it's personal, one might like the wine while other might hate it. Thus, wine shoppers prefer good quality wine which make this question whether is it possible to predict quality of a wine.

2. Problem Definition

Based on the given dataset (section 3 for more details), these are the question that I want to answer in this project:

- (1) Can we classify whether wine is good or bad based on quality of a wine?
- (2) Can we create a regression model to predict the quality of the given wine?

3. Dataset

The dataset that I used for this project is obtained from UC Irvine Machine Learning Repository¹. The dataset included two subsets related to red and white Vinho Verde wine, from the north of Portugal. The dataset has 11 features such as citric acid, chlorides, density, pH, etc. which are obtained from physicochemical and one output variable which is quality of the wine obtained from sensory data. Due to privacy and logistic issues, there is no data about grape types, wine brand, wine selling price, etc. In this project we focused on red wine dataset which has 1599 instances.

4. Data Acquisition

Problem type can be both regression and classification. Here's the reason:

- Regression: because the target variable quality is a numeric score (from 0 to 10).

- Classification: We convert the quality column (a score from 0–10) into discrete quality categories.

Feature Name	Type	Description
fixed acidity	float	Non-volatile acids (mainly tartaric acid) present in wine
volatile acidity	float	Acetic acid content; high levels can lead to an unpleasant vinegar taste
citric acid	float	A weak organic acid that can add freshness and flavor to wines
residual sugar	float	Sugar remaining after fermentation (g/L)
chlorides	float	Amount of salt (sodium chloride) in wine
free sulfur dioxide	float	SO ₂ in free form; helps prevent microbial growth and oxidation
total sulfur dioxide	float	Total SO ₂ , including bound and free forms
density	float	Density of wine; correlated with alcohol and sugar content
pH	float	Acidity level of wine (inverse log of hydrogen ion concentration)
sulphates	float	Sulfur dioxide derivatives; a wine preservative
alcohol	float	Alcohol percentage by volume (%)

Table 1. Feature names, types, and descriptions for the Red Wine Quality dataset.

The output which is quality, can be:

- Int, original wine quality score (0–10), given by human tasters

¹ <https://archive.ics.uci.edu/dataset/186/wine+quality>

² <http://www3.dsi.uminho.pt/pcortez/wine/>

- Label, categorical like low, medium, high

5. Statistical Analysis

In this section, we will use basic statistics and visuals to understand our data. The mean (average) and median (middle value) help us identify typical values in the dataset, while the standard deviation tells us how much the data varies. A box plot summarizes the data's range and flags the unusual points (outliers), and a histogram shows how values are distributed, like whether the most are clustered or spread out. Finally, a heatmap reveals relationships between features. Together, these tools guide us in cleaning the data, choosing the right model, and making reliable predictions.

5.1 Mean

The mean of a feature is the average of all its values. For a dataset with n instances, where x_i represents the i -th value of the feature, the mean $\bar{x}$ is computed as the sum of all x_i divided by n :

$$\bar{x} = \frac{\sum_{i=1}^n x_i}{n}$$

Feature Name	Feature Mean
fixed acidity	8.319637
volatile acidity	0.527821
citric acid	0.270976
residual sugar	2.538806
chlorides	0.087467
free sulfur dioxide	15.874922
total sulfur dioxide	46.467792
density	0.996747
pH	3.311113
sulphates	0.658149
alcohol	10.422983
quality	5.636023

Table 2. Computed mean values for each feature.

5.2 Median

The median of a feature is the middle value in an ordered (ascending or descending) dataset. When the number of instances (n) is odd, the median is the central value that divides the data into two equal halves. For even n , where there is no single middle value exists, the median is computed as the average of two central values.

$$\text{Median} = \begin{cases} \frac{(n+1)^{th}}{2} \text{ term}, & n \text{ is odd} \\ \frac{\frac{n}{2}^{th} \text{ term} + (\frac{n}{2} + 1)^{th} \text{ term}}{2}, & n \text{ is even} \end{cases}$$

Feature Name	Feature Median
fixed acidity	7.9
volatile acidity	0.52
citric acid	0.26
residual sugar	2.2
chlorides	0.079
free sulfur dioxide	14
total sulfur dioxide	38
density	0.99675
pH	3.31
sulphates	0.62
alcohol	10.2
quality	6

Table 3. Computed median values for each feature.

5.3 Standard Deviation

The spread of a dataset is measured by the standard deviation (SD). The SD is derived from the variance, which is the covariance of a variable with itself ($\text{Cov}(x, x)$). Mathematically, the standard deviation ($s(x)$) is the square root of the variance:

$$s = \sqrt{\text{Var}(x)} = \sqrt{\text{COV}(x, x)}$$

For a sample of n instances with arithmetic mean $\bar{x}$, and the i -th individual value in the dataset x_i , the standard deviation calculated as:

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Feature Name	Feature Standard Deviation
fixed acidity	1.741096
volatile acidity	0.179060
citric acid	0.194801
residual sugar	1.409928
chlorides	0.047065

free sulfur dioxide	10.460157
total sulfur dioxide	32.895324
density	0.001887
pH	0.154386
sulphates	0.169507
alcohol	1.065668
quality	0.807569

Table 4. Computed standard deviation values for each feature.

5.4 Boxplot

A boxplot is a graphical representation of the distribution of a dataset. It displays key summary statistics such as the median, quartiles, and potential outliers in a concise and visual manner. Below are the boxplots for each feature:

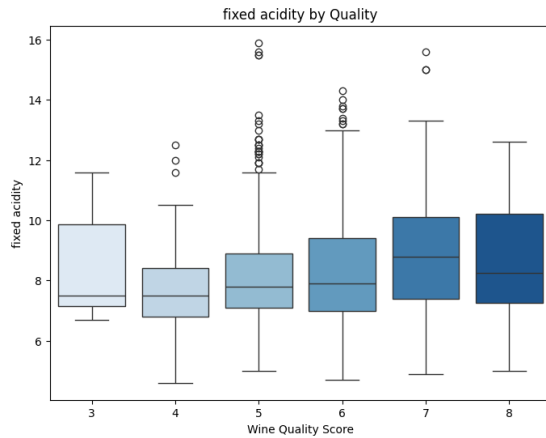


Figure 1. Boxplot of fixed acidity across quality scores. The median is relatively stable but slightly increases as the quality increases. The interquartile range (IQR) remains narrow across all scores, but there are several high-value outliers, especially in lower quality wines.

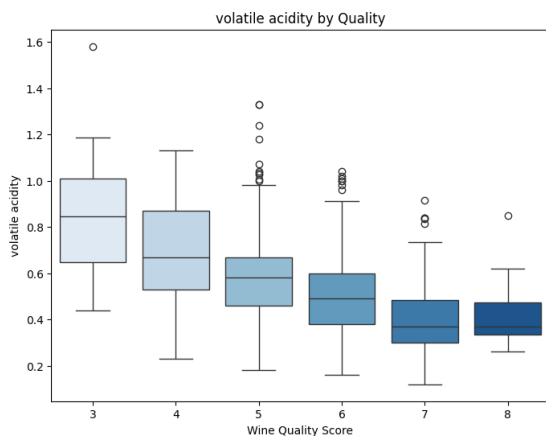


Figure 2. Volatile acidity tends to decrease as wine quality increases. Since volatile acidity is associated with vinegar-like flavor, this inverse trend shows lower volatile acidity contributes to better tasting wines.

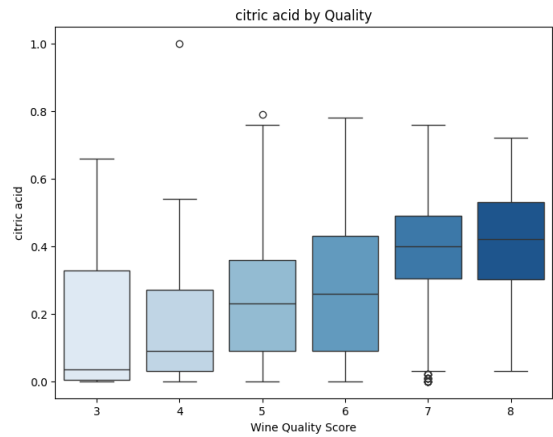


Figure 3. Citric acid tends to increase with higher quality scores. The median value rises steadily, especially for the wines rated 4 and above. This implies citric acid may be a positive influence on quality.

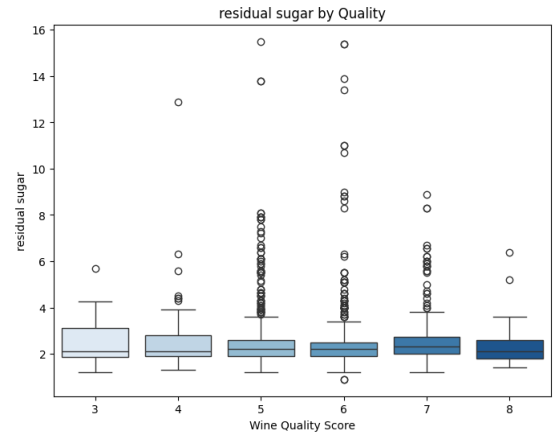


Figure 4. Residual sugar levels are mostly consistent across all quality scores. The medians are nearly aligned, and although there are some outliers, the overall distribution shows that residual sugar is not a strong discriminator for wine quality in this dataset.

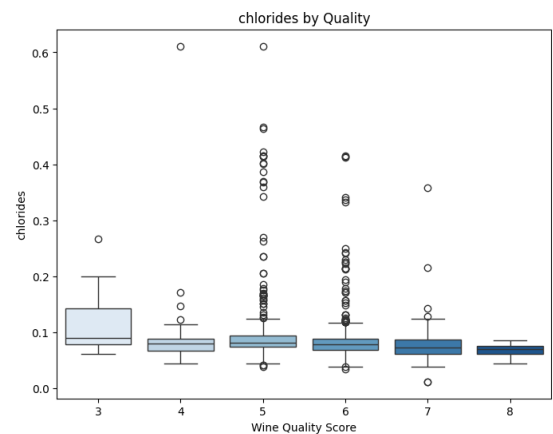


Figure 5. Chloride levels show a slight decrease as the quality increases. Lower quality wines have higher medians and a wide range, whereas higher quality wines show lower and tighter distribution.

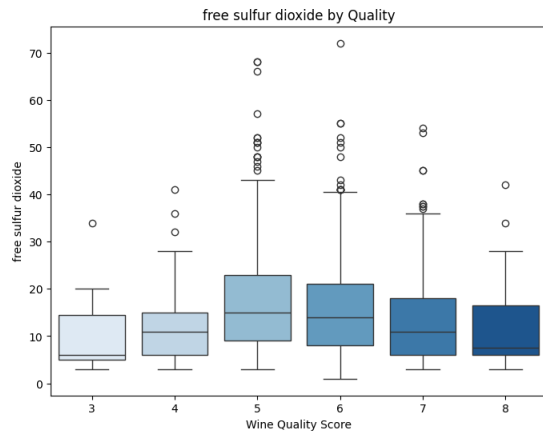


Figure 6. Free sulfur dioxide shows a high variance across all quality levels with no clear increasing or decreasing level. The presence of outliers and the wide IQR across all groups imply that this feature has a weak or non-linear relationship with win quality.

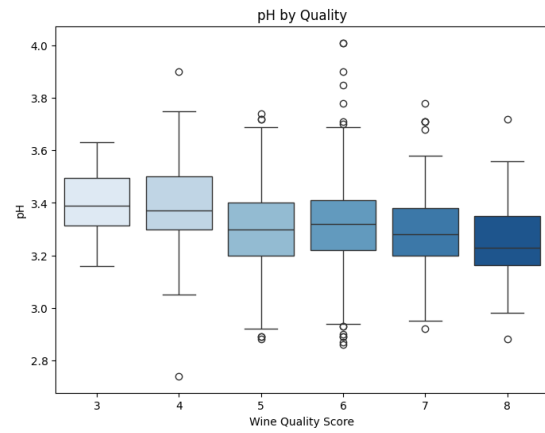


Figure 9. pH levels across wine qualities are quite consistent, with only minor variation. There is no obvious trend, and medians remain in a narrow range. This suggests that pH may not be a major factor in determining quality.

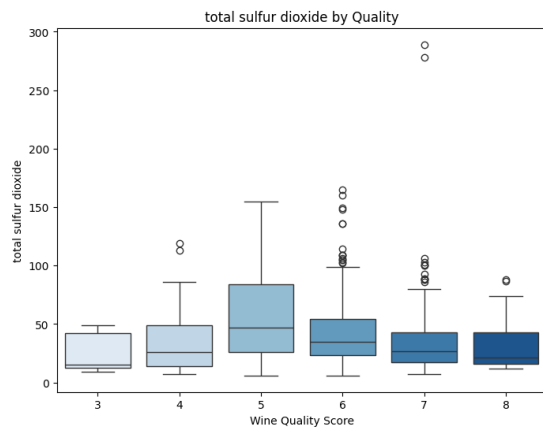


Figure 7. Total sulfur dioxide shows a decreasing trend with increasing wine quality. Wines with lower quality have significantly higher median, while higher quality wines tend to have lower levels. This supports the idea that too much sulfur dioxide may reduce wine quality.

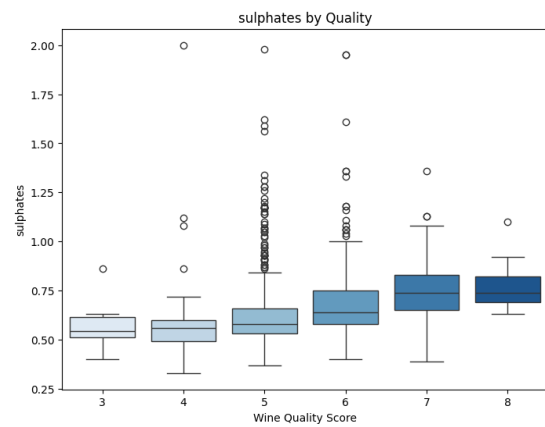


Figure 10. Sulphate levels show a clear positive trend with wine quality. High quality wines have increased median values and tighter distribution, implying that sulphate may enhance stability and flavor, thereby improving the quality.

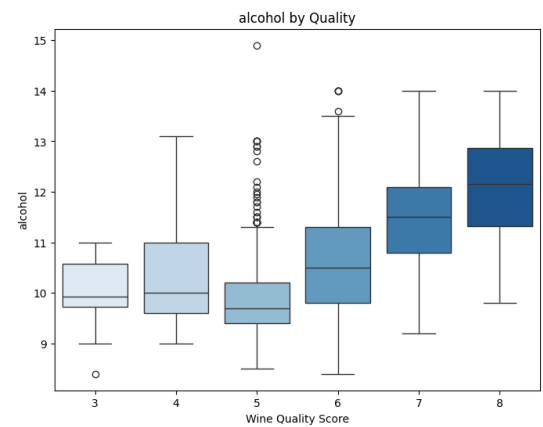


Figure 8. Density appears to slightly decrease with wine quality. Since alcohol lowers the density of a liquid, this might correlate with the higher alcohol content seen in better wines.

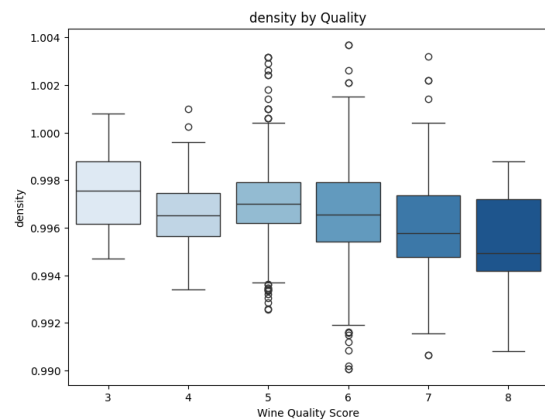


Figure 11. This is one of the clearest trends, Alcohol increases with the wine quality. Each step up in quality tends to show a higher median alcohol level. Higher alcohol content can enhance flavor, which contributes positively to wine ratings.

5.5 Histogram

A histogram is a bar graphed-like representation of the distribution of a variable. It shows the frequency of occurrences of each value of the variable. The x-axis represents the range of values of the variable, and the y-axis represents the frequency or count of each value.

The histogram graphically shows the following:

- Frequency of different data points in the dataset.
- Location of the center of the data.
- The spread of the dataset.
- Variance of dataset.
- Presence of outliers in the dataset.
- Identifying patterns, trends, variability and other characteristics of the dataset.

Here's the histogram of each variable:

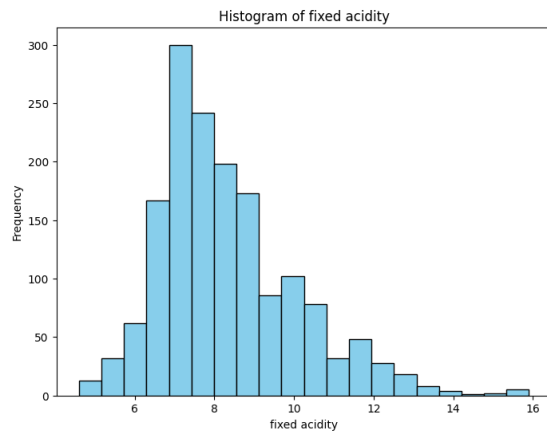


Figure 12. Fixed acidity values exhibit a unimodal distribution with a slightly right-skewed shape. The most common values fall between 6 and 8 g/dm³, indicating a central tendency around 7. This suggests that extreme acidity levels are rare and most wines tend to cluster in around a moderately acidic profile.

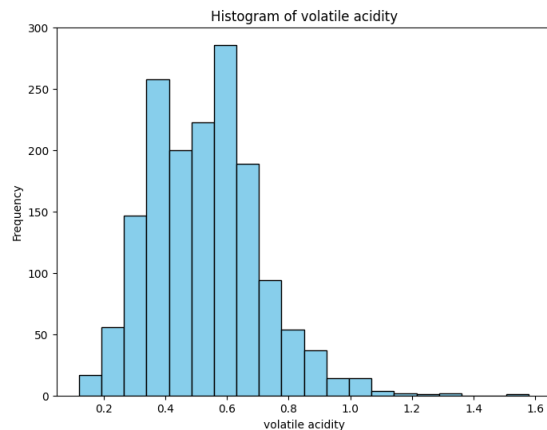


Figure 13. Volatile acidity also shows a slightly right-skewed distribution with a peak between 0.3 and 0.7 g/dm³. Very high acidity (above 1.0) is uncommon. Since volatile acidity is related to spoilage, the dataset reflects the fact that most wines maintain the acceptable levels.

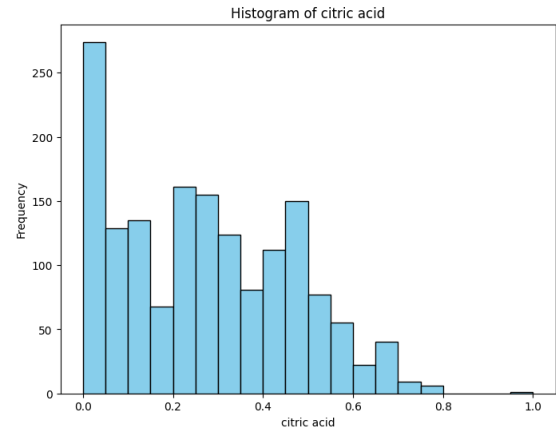


Figure 14. Citric acid shows a heavily right-skewed, with many samples near 0, and fewer as value increases. This indicates that a large number of wines contains little to no citric acid. Few values near 1.0 may be seen as outliers.

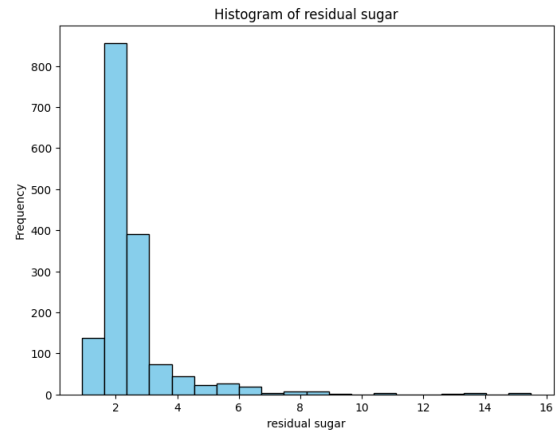


Figure 15. Residual sugar is heavily skewed to the right, with an extremely high concentration of values near 2 g/dm³. Although most wines have low sugar content, a long tail extends beyond 10 and up to 15 g/dm³, showing the presents of outliers, likely due to specific wine styles or production method.

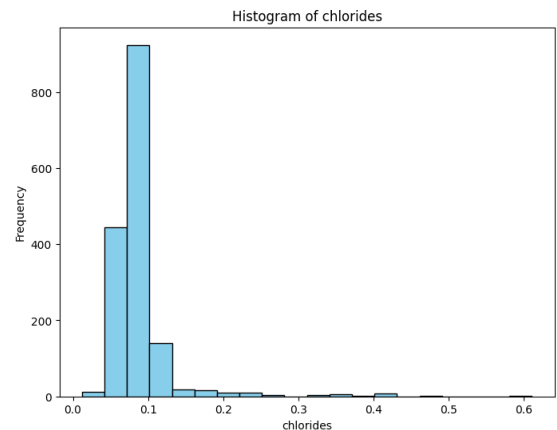


Figure 16. The histogram of chlorides is sharply right-skewed, concentrated around 0.05 to 0.1 g/dm³. Chloride values above 0.2 are very rare. Most wines contain a low concentration of self-related compounds.

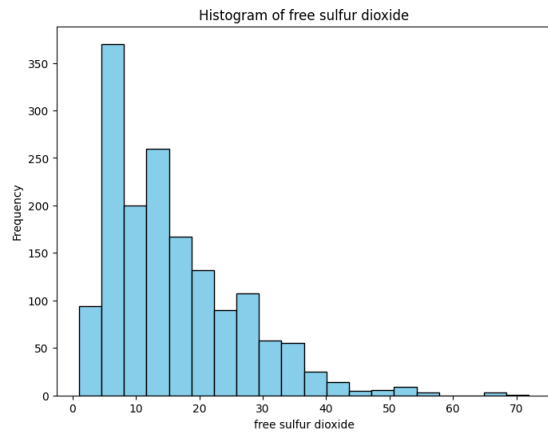


Figure 17. The distribution of free sulfur dioxide is right-skewed, with the majority of samples below 30 mg/dm³. Most wines have low moderate level with central tendency near 15 mg/dm³. The presence of values above 50 mg/dm³, indicates outliers that may reflect wines.

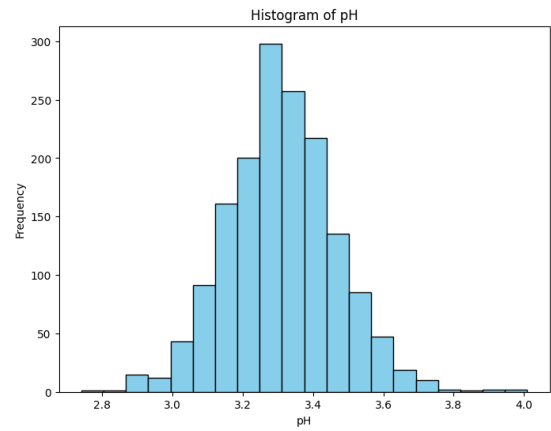


Figure 20. The histogram of pH shows a near-normal (bell-shaped) distribution, centered about 3.3. Values below 2.9 or above 3.8 might be considered mild outliers, though the spread remains quite natural for a biological measurement like pH.

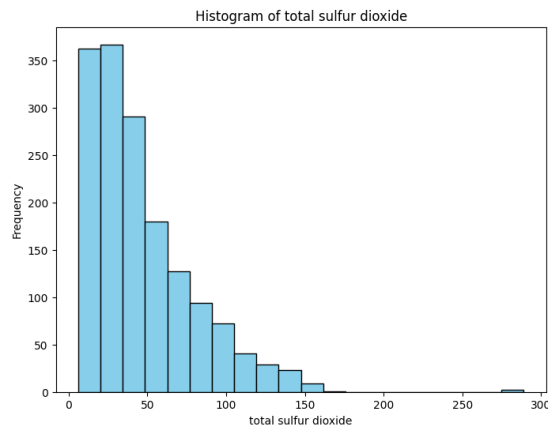


Figure 18. Total sulfur dioxide also has a right-skewed distribution, with most values between 0 and 150 mg/dm³, and a central tendency around 50-60. A noticeable number of values above 200 are clearly outliers, perhaps pointing to exceptions in preservation methods.

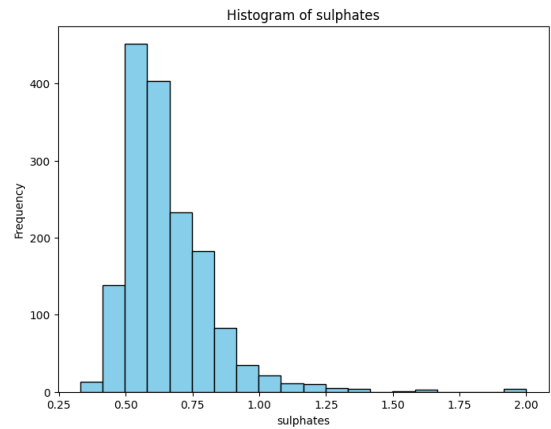


Figure 21. Sulphates content distribution is right-skewed, with the most value falling between 0.5 and 0.75 g/dm³. The central tendency is around 0.6. a few values above 1.2 appear as outliers and could correspond to wines with more intense preservation.

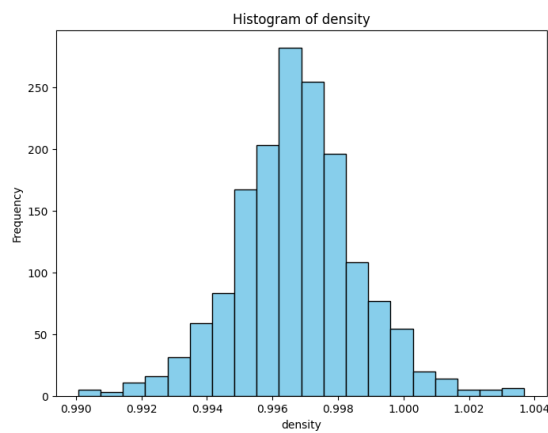


Figure 19. Density is uniquely normally distributed, centered about 0.996 to 0.997 g/cm³. There are no obvious outliers, indicating that most wines in the dataset have very similar density values, likely reflecting similar sugar and alcohol contents.

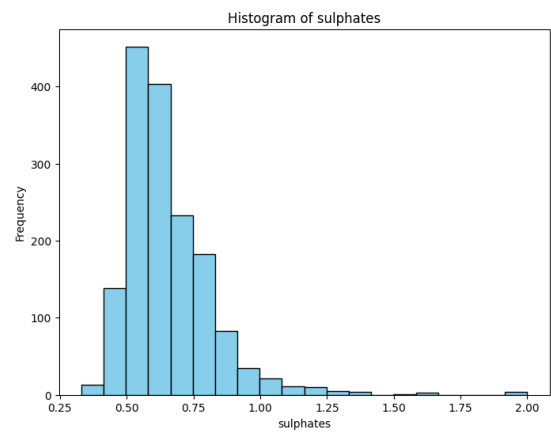


Figure 22. Sulphates presents a slightly right-skewed distribution, with the most values falling between 0.4 and 0.8 g/dm³. The central tendency is near 0.6. A few values above 1.2 appear as outliers and could correspond to wines with more intense preservation or flavor enhancement.

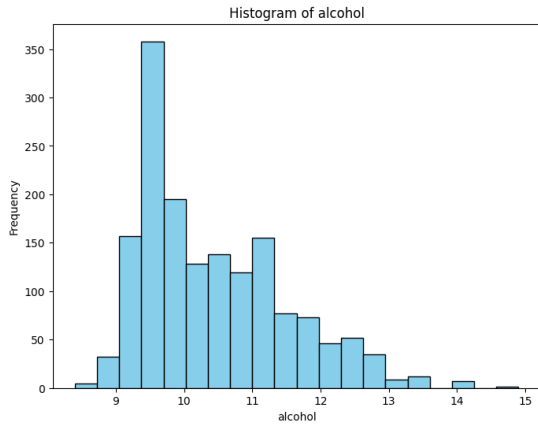


Figure 23. Alcohol displays a slightly right-skewed distribution with central concentration around 10%. Most wines fall between 9 to 12% range. This feature shows moderate spread and variance, with likely outliers corresponding to wines with extended fermentation.

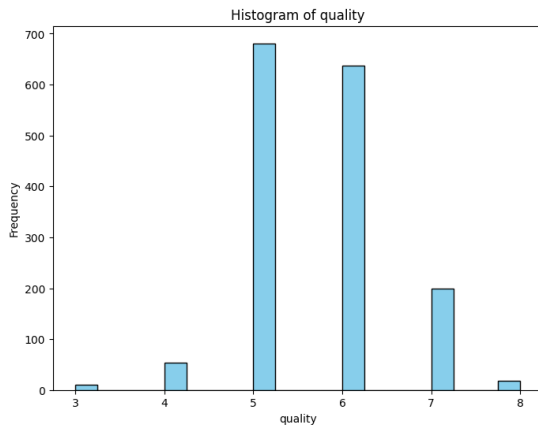


Figure 24. Quality, unlike the continues features, is a discrete variable, typically ranging from 3 to 8. The most frequent scores are 5 and 6, indicating that most wines are rated as average or slightly above average in quality. The distribution is unimodal with slightly left-skewed, with very few wines receiving low or high scores. Ratings of 3 and 8 may be considered as outliers.

5.6 Correlation heatmap

A correlation heatmap is a 2D graphical representation of a correlation matrix between multiple variables. It uses colored cells to indicate correlation values, making patterns and relationships within data visually interpretable. The color intensity of each cell represents the strength of the correlation.

- A value of 1 indicates a strong positive relationship between two variables (dark colors).
- A value of 0 suggests no relationship between them (neutral colors).
- A value of -1 signals a strong negative or inverse relationship (light colors).

By using correlation matrix, we can analyze and visualize the connection in our data. This makes it an essential step

before building machine learning models, because understanding which variables are correlated, helps us identify the most influential factors for our model.

The following formula is used to compute the correlation coefficient between two variables:

$$\text{Cor}(x, y) = \frac{n(\sum xy) - (\sum x)(\sum y)}{\sqrt{[n\sum x^2 - (\sum x)^2][n\sum y^2 - (\sum y)^2]}}$$

Or with using covariance we have:

$$\text{Cor}(x, y) = \frac{\text{Cov}(x, y)}{s_x s_y}$$

Where n is number of observations, $\sum x$ is sum of the observations of the first variable, $\sum y$ is sum of the observations of the second variable, $\sum xy$ is sum of the product of each pair of corresponding observations of the two variables, $\text{Cov}(x, y)$ is the covariance, s_x is the standard deviation of the first variable, and s_y is the standard deviation of the second variable.

Here's the correlation heatmap of the dataset:

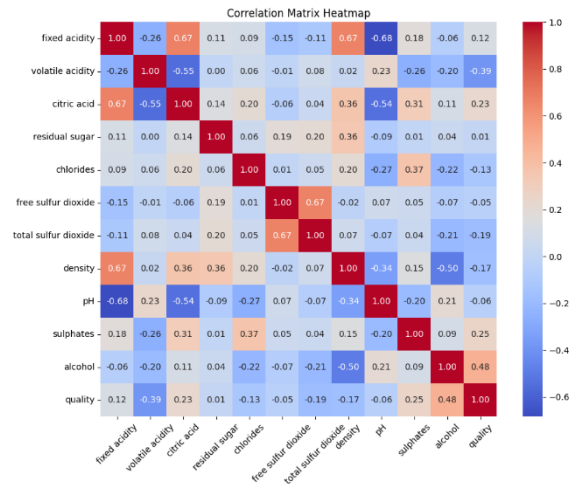


Figure 25. Correlation heatmap, which visualize Pearson correlation coefficients between all pairs of features in the dataset.

5.6.1 Relationships with quality (Target Variable)

- Alcohol shows the strongest positive correlation with quality ($r = 0.48$), indicating that wines with higher alcohol content tend to receive higher quality scores.
- Volatile acidity has a moderate negative correlation with quality ($r = -0.39$), meaning that as volatile acidity increases (which can give wine a vinegary taste), perceived quality decreases.
- Sulphates are positively correlated with quality ($r = 0.25$), though less strongly than alcohol. Sulphates help keep microbes stable, which may influence sensory quality.

- Others features show a very weak correlation with quality (mostly between -0.2 and 0.2), suggesting that they are less predictive individually.

5.6.2 Strong correlation between features

- Free sulfur dioxide and total sulfur dioxide are strongly positively correlated ($r = 0.67$), which make sense because total SO_2 includes free SO_2 .
- Fixed acidity and density also show a strong positive correlation ($r = 0.67$), likely because higher acidity increases density.
- Density is strongly negatively correlated with alcohol ($r = -0.5$). Alcohol is less dense than water, so as alcohol content increases, density decreases.
- Fixed acidity and citric acid are positively correlated ($r = 0.67$), which is logical since citric acid one of the acids that makes up fixed acidity.
- Citric acid and volatile acidity have a negative correlation ($r = -0.55$). Wines with more citric acid tend to have lower volatile acidity.

5.6.3 Notable negative correlations

- pH is strongly negatively correlated with fixed acidity ($r = -0.68$) and citric acid ($r = -0.54$). This is expected since pH and acidity are inversely related, higher acidity means lower pH.
- Density is also negatively correlated with pH ($r = -0.34$), suggesting denser wines tends to have lower pH (they are more acidic).

5.6.4 Weak or no correlations

Many variables show very weak correlation with others (close to zero), such as:

- Residential sugar and quality ($r = -0.01$).
- Chlorides and quality ($r = -0.13$).
- Free sulfur dioxide and quality ($r = -0.05$).

These features may not directly impact wine quality or may do in non-linear ways not captured by Pearson correlation.

6. Data Preprocessing

Preprocessing is a critical step in preparing the dataset for machine learning tasks, ensuring data quality, compatibility with algorithms, and optimal model performance. Two distinct tasks were performed:

Regression: Predicting the numerical quality score (0–10) using continuous features.

Classification: Predicting discretized quality categories ('Bad': 0–4, 'Medium': 5–6, 'Good': 7–10) with additional discretized input features.

Separate preprocessing pipelines were developed for each task to align with their requirements. For regression, the focus was on preserving continuous feature information, while for classification, discretization and encoding were introduced to create categorical labels. The preprocessing steps included handling missing values, discretization (classification only), encoding categorical features (classification only), normalization/standardization, and splitting the data into training and testing sets. All preprocessing was implemented using Python with libraries such as pandas, NumPy, and scikit-learn, and the preprocessed data was saved as CSV files for use in model training.

6.1 Preprocessing for Regression

The regression task aims to predict the numerical quality score, requiring continuous features to maximize predictive accuracy. The preprocessing pipeline was designed to maintain the granularity of the dataset's numerical features while ensuring compatibility with regression algorithms.

6.1.1 Handling Missing Values

Purpose:

Missing values can disrupt model training and bias predictions. This step ensures a complete dataset by identifying and addressing any missing entries.

Methodology:

The dataset was inspected for missing values, and imputation was applied as a precautionary measure to handle potential missing data, using a statistical approach suitable for numerical features.

Implementation Details:

The dataset was loaded using pandas' `read_csv` function, with features separated by semicolons.

Missing values were checked using `data.isnull().sum()`, confirming no missing values in any of the 11 input features or the target quality, consistent with the dataset's documentation.

A `SimpleImputer` from scikit-learn with a mean strategy was applied to impute any missing values by replacing them with the mean of the respective feature. Since no missing values were present, this step had no effect but ensured pipeline robustness.

The imputed data was stored in a new `DataFrame`, `data_imputed`, preserving the original 11 features and quality.

Considerations:

The absence of missing values simplified this step, but including imputation ensures the pipeline's generalizability to other datasets with potential missing data. Mean imputation was chosen for numerical features to preserve central tendency.

6.1.2 Discretization

No discretization was applied, as regression algorithms benefit from the full range of continuous features, which capture fine-grained relationships with the numerical quality target.

6.1.3 Encoding

Not applicable, as the dataset contained only numerical features after skipping discretization.

6.1.4 Normalization/Standardization**Purpose:**

Normalization and standardization scale numerical features to ensure equal contribution during model training, particularly for scale-sensitive algorithms.

Methodology:

Standardization transforms features to have zero mean and unit variance, while normalization scales features to a [0,1] range. Standardization was chosen for the primary pipeline, with normalization provided as an alternative.

Implementation Details:

Features were separated into input (X , all 11 features excluding quality) and target (y , quality).

StandardScaler from scikit-learn was applied to standardize X , resulting in $X_{\text{standardized}}$ with zero mean and unit variance for each feature. This ensures compatibility with scale-sensitive algorithms.

MinMaxScaler was applied as an alternative, producing $X_{\text{normalized}}$ with features scaled to [0,1], suitable for testing with algorithms like Support Vector Regressor.

The standardized features ($X_{\text{standardized}}$) were used for the final pipeline, stored as a DataFrame with original column names.

Considerations:

Standardization was prioritized due to its suitability for algorithms assuming normally distributed data. Normalization was included for flexibility, as Support Vector Regressor with an RBF kernel may perform differently with [0,1]-scaled features. The dataset's features have varying scales (e.g., alcohol in % vs. total sulfur dioxide in mg/L), making scaling essential.

6.1.5 Train/Test Split**Purpose:**

Splitting the data into training and testing sets enables model training on one subset and evaluation on unseen data, assessing generalization performance.

Methodology:

A standard 80/20 split was used, with 80% of the data for training and 20% for testing, ensuring sufficient data for learning and evaluation.

Implementation Details:

The standardized features ($X_{\text{standardized}}$) and target (y) were split using scikit-learn's `train_test_split` with `test_size=0.2` and `random_state=42` for reproducibility.

This resulted in:

- Training set: 1279 samples (X_{train} , y_{train}).
- Test set: 320 samples (X_{test} , y_{test}).

The preprocessed data was saved as CSV files: `X_train_reg.csv`, `X_test_reg.csv`, `y_train_reg.csv`, `y_test_reg.csv`.

Considerations:

The 80/20 split balances training data availability with test set size, suitable for the dataset's 1599 samples. The fixed `random_state` ensures consistent splits across experiments, enabling fair model comparisons.

6.2 Preprocessing for Classification

The classification task predicts discretized quality categories ('Bad', 'Medium', 'Good'), requiring the creation of categorical labels and handling class imbalance. The preprocessing pipeline was designed to support classification algorithms by discretizing features and encoding them appropriately.

6.2.1 Handling Missing Values**Purpose:**

Ensures a complete dataset by addressing any missing values, preventing errors in model training.

Methodology:

Identical to regression, the dataset was inspected, and imputation was applied to handle potential missing data.

Implementation Details:

Missing values were checked using `data.isnull().sum()`, confirming none were present.

`SimpleImputer` with a mean strategy was applied, producing `data_imputed` with no changes due to the absence of missing values.

The imputed DataFrame retained all original features and quality.

Considerations:

The lack of missing values simplified this step, but imputation was included for pipeline robustness, consistent with the regression pipeline.

6.2.2 Discretization**Purpose:**

Discretization transforms continuous features into categorical bins to create class labels for classification and simplify patterns for certain algorithms (e.g., Decision Trees).

Methodology:

The target quality was discretized into categorical labels, and two influential input features (alcohol, residual sugar) were binned to capture non-linear effects and enhance model interpretability.

Implementation Details:

Target Discretization: The quality feature (0–10) was binned into three categories using pandas' `pd.cut`:

- 'Bad: 0–4 (poor quality).
- 'Medium': 5–6 (average quality, most frequent).
- 'Good: 7–10 (excellent quality).

Bins: [0, 4, 6, 10], labels: ['Bad', 'Medium', 'Good'], with `include_lowest=True`. The new column `quality_binned` was added to `data_imputed`.

Input Feature Discretization:

- **Alcohol:** Binned into 'Low' (0–10%), 'Medium' (10–12%), 'High' (12–15%) using bins [0, 10, 12, 15], reflecting typical wine alcohol ranges. Result: `alcohol_binned`.
- **Residual Sugar:** Binned into 'Low' (0–2 g/dm³), 'Medium' (2–4 g/dm³), 'High' (4–20 g/dm³) using bins [0, 2, 4, 20], accounting for its skewed distribution. Result: `residual_sugar_binned`.

Considerations:

Discretizing quality was essential to create classification labels. alcohol and residual sugar were chosen for discretization due to their influence on wine quality and non-linear relationships, as identified in exploratory analysis. Other features (e.g., volatile acidity) could be discretized but were kept continuous to balance dimensionality and information retention.

6.2.3 Encoding Categorical Features**Purpose:**

Converts categorical features into numerical representations for model compatibility, as machine learning algorithms require numerical inputs.

Methodology:

Label Encoding was used for the target to create integer class labels, and One-Hot Encoding was applied to discretized input features to avoid implying ordinal relationships.

Implementation Details:

- Label Encoding: The `quality_binned` column was encoded using scikit-learn's `LabelEncoder`, mapping 'Bad' to 0, 'Medium' to 1, and 'Good' to 2, creating `quality_binned_encoded` as the classification target.
- One-Hot Encoding: The discretized input features (`alcohol_binned`, `residual_sugar_binned`) were encoded using pandas' `get_dummies`, creating binary columns (e.g., `alcohol_Low`, `alcohol_Medium`, `alcohol_High`, `sugar_Low`, etc.). This expanded the feature set to include these binary columns alongside the original numerical features.

The resulting DataFrame, `data_onehot`, contained the original 11 numerical features (minus quality), the encoded target, and the one-hot encoded columns.

Considerations:

Label Encoding was suitable for the target, as classification algorithms handle integer labels appropriately. One-Hot Encoding for input features was critical for algorithms like Logistic Regression and Support Vector Classifier, which are sensitive to ordinal assumptions.

6.2.4 Normalization/Standardization**Purpose:**

Scales features to ensure equal contribution, particularly for scale-sensitive algorithms like Logistic Regression and Support Vector Classifier.

Methodology:

Standardization was applied to transform features to zero mean and unit variance, with normalization provided as an alternative for flexibility.

Implementation Details:

Features (X) were defined as all columns in `data_onehot` except `quality_binned_encode`, `quality_binned` and, `quality`, including original numerical features and one-hot encoded columns. The target (y) was `quality_binned_encode`.

StandardScaler was applied to X , producing $X_{\text{standardized}}$ with standardized features, ensuring compatibility with scale-sensitive classifiers.

MinMaxScaler was applied to create $X_{\text{normalized}}$ with features scaled to $[0, 1]$, suitable for testing with Support Vector Classifier.

The standardized features ($X_{\text{standardized}}$) were used for the final pipeline, stored as a DataFrame.

Considerations:

Standardization was prioritized due to its compatibility with Logistic Regression and Support Vector Classifier, which assume normally distributed data. The one-hot encoded features (binary 0/1) were also standardized, which is acceptable as it does not disrupt their interpretation. Normalization was included for potential optimization with Support Vector Classifier's RBF kernel.

6.2.5 Train/Test Split

Purpose:

Splits the data into training and testing sets to enable model training and evaluation on unseen data.

Methodology:

An 80/20 split was used, consistent with the regression pipeline, to ensure sufficient training data and a robust test set.

Implementation Details:

The standardized features ($X_{\text{standardized}}$) and target (y , `quality_binned_encode`) were split using `train_test_split` with `test_size=0.2` and `random_state=42`.

This resulted in:

- Training set: 1279 samples (X_{train} , y_{train}).
- Test set: 320 samples (X_{test} , y_{test}).

The preprocessed data was saved as CSV files: `X_train_clf.csv`, `X_test_clf.csv`, `y_train_clf.csv`, `y_test_clf.csv`.

Considerations:

The consistent split and `random_state` ensure comparability with regression results. The test set size (320 samples) is sufficient to evaluate classification performance, despite the class imbalance ('Bad' class is underrepresented).

6.3 Summary

The preprocessing pipelines for regression and classification is shown in the following table:

Task	Regression	Classification
Missing Values	Mean Imputation	Mean Imputation
Discretization	None	quality, alcohol, residual_sugar
Encoding	None	Label + One-Hot
Scaling	StandardScaler	StandardScaler
Train/Test	80/20	80/20

7. Choose and Train Models

This section evaluates the performance of ten machine learning models applied to the dataset. The dataset was preprocessed (section 6 for more details)

Five regression algorithms and five classification algorithms were implemented. The regression models are evaluated using Mean Squared Error (MSE), Mean Absolute Error (MAE), and R^2 Score, while classification models are evaluated using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

7.1 Regression Models

The regression models predict the numerical quality score. Performance is evaluated using Mean Squared Error (MSE), Mean Absolute Error (MAE), and R^2 Score, as these metrics are standard for regression tasks:

Mean Squared Error (MSE): Measures the average squared difference between predicted and actual values, emphasizing larger errors. Lower MSE indicates better accuracy.

MSE provides a way to quantify how much error exists in a model's predictions, and is particularly useful in fields like machine learning, data science, and statistics.

$$MSE = \frac{1}{n} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- y_i is the actual value (true value).
- $\hat{y}_i$ is the predicted value (from the model).
- n is the total number of data points.

Mean Absolute Error (MAE): Measures the average absolute difference, providing a straightforward measure of prediction error in the same units as quality (0–10). Lower MAE is better.

$$MAE = \frac{1}{n} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- y_i is the actual value for the i -th observation.
- $\hat{y}_i$ is the predicted value for the i -th observation.
- n is the total number of observations.

R² Score: Represents the proportion of variance in quality explained by the model, ranging from 0 to 1 (higher is better). An R² close to 1 indicates a good fit.

We calculate R-Square in the following steps:

1. First, calculate the mean of the target/dependent variable y and we denote it by $\bar{y}$.
2. Calculate the total sum of squares by subtracting each observation y_i from $\bar{y}$, then squaring it and summing these square differences across all the values. It is denoted by

$$SST = \sum_{i=1}^n (y_i - \bar{y})^2$$

3. We estimate the model parameter using a suitable regression model such as Linear Regression or SVM Regressor.
4. We calculate the Sum of squares due to regression which is denoted by SSR. This is calculated by subtracting each predicted value of y denoted by $\hat{y}_i$ from y_i squaring these differences and then summing all the n terms.

$$SSR = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$$

5. We calculate the sum of squares (SSE). It explains unaccounted variability in the dependent y after predicting these values from an independent variable in the model.

$$SSE = \frac{1}{n} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

6. We can then use either

$$R^2 = \frac{SSR}{SST}$$

or

$$R^2 = 1 - \frac{SSE}{SST}$$

7.1.1 Linear Regression

Results:

- $MSE = 0.3900$

The average squared error corresponds to a root mean squared error (RMSE) of ~ 0.62 quality points ($\sqrt{0.39}$). This indicates moderate prediction errors, as errors of ~ 0.6 points are reasonable given the 0–10 scale and clustered scores.

- $MAE = 0.5035$

On average, predictions deviate by ~ 0.50 quality points, which is acceptable but indicates that errors are noticeable, especially for a dataset where scores differ by 1–2 points.

- $R^2 = 0.4032$

The model explains $\sim 40.3\%$ of the variance in quality, which is modest. This suggests that Linear Regression captures some relationships but struggles with the dataset's non-linear patterns.

Conclusion:

The performance is moderate. Linear Regression serves as a baseline but is suboptimal for this dataset due to its non-linear nature.

7.1.2 Decision Tree Regressor

Results:

- $MSE = 0.6219$

The high MSE (RMSE ~ 0.79) indicates larger prediction errors compared to other models, with errors averaging ~ 0.79 quality points. This suggests poor accuracy.

- $MAE = 0.4656$

The MAE (~ 0.47) is lower than the RMSE, indicating that errors are somewhat consistent but still significant. This is better than the MSE suggests, as it's less sensitive to outliers.

- $R^2 = 0.0484$

The extremely low R² ($\sim 4.8\%$) indicates that the model explains almost none of the variance in quality, performing barely better than a mean-based baseline.

Conclusion:

The performance is poor. Decision Tree Regressor is unsuitable due to its poor generalization and low explanatory power.

7.1.3 Random Forest Regressor

Results:

- $MSE = 0.3014$

The lowest MSE among the models (RMSE ~ 0.55) indicates high accuracy, with errors averaging ~ 0.55 quality points, which is small for the 0–10 scale.

- $MAE = 0.4220$

The lowest MAE shows average errors of ~ 0.42 points, an excellent result indicating precise predictions.

$$- R^2 = 0.5389$$

The model explains $\sim 53.9\%$ of the variance, the highest among the models, reflecting a strong fit to the data.

Conclusion:

The performance is strong. Random Forest Regressor is highly suitable due to its accuracy and ability to handle complex patterns.

7.1.4 Support Vector Regressor (SVR)

Results:

$$- MSE = 0.3516$$

The MSE (RMSE ~ 0.59) indicates moderate errors, better than Decision Tree but worse than Random Forest, with errors averaging ~ 0.59 points.

$$- MAE = 0.4540$$

The MAE (~ 0.45) shows average errors comparable to Random Forest, suggesting good predictive accuracy.

$$- R^2 = 0.4620$$

The model explains $\sim 46.2\%$ of the variance, better than Linear Regression but below Random Forest, indicating a decent but not optimal fit.

Conclusion:

The performance is good but not exceptional. SVR is suitable but requires hyperparameter tuning to compete with ensemble methods.

7.1.5 Gradient Boosting Regressor

Results:

$$- MSE = 0.3644$$

The MSE (RMSE ~ 0.60) indicates moderate errors, slightly worse than SVR but better than Decision Tree, with errors averaging ~ 0.60 points.

$$- MAE = 0.4853$$

The MAE (~ 0.49) is higher than Random Forest and SVR, indicating slightly larger average errors.

$$- R^2 = 0.4424$$

The model explains $\sim 44.2\%$ of the variance, comparable to SVR but below Random Forest, suggesting a decent fit.

Conclusion:

The performance is good but not the best. Gradient Boosting is suitable but underperforms compared to Random Forest in this case.

7.2 Classification Models

The classification models predict discretized quality categories ('Bad': 0–4, 'Medium': 5–6, 'Good': 7–10).

The evaluation metrics are:

Confusion Matrix: It is a matrix of size 2×2 for binary classification with actual values on one axis and predicted on another.

		Actual	
		Negative	Positive
Prediction	Negative	True Negative	False Negative
	Positive	False Positive	True Positive

True Positive (TP): model correctly predicts the positive class (prediction and actual both are positive).

True Negative (TN): model correctly predicts the negative class (prediction and actual both are negative).

False Positive (FP): model gives the wrong prediction of the negative class (predicted-positive, actual-negative). FP is also called a TYPE I error.

False Negative (FN) – model wrongly predicts the positive class (predicted-negative, actual-positive). FN is also called a TYPE II error.

Accuracy: Proportion of all classifications that were correct, whether positive or negative. It is mathematically defined as:

$$Accuracy = \frac{\text{correct classifications}}{\text{total classifications}} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision: Proportion of all the model's positive classifications that are actually positive. It is mathematically defined as:

$$Precision = \frac{\text{correctly classified actual positive}}{\text{everything classified as positive}} = \frac{TP}{TP + FP}$$

Recall: The true positive rate (TPR), or the proportion of all actual positives that were classified correctly as positives. It is mathematically defined as:

$$Recall = \frac{\text{correctly classified actual positive}}{\text{all actual positives}} = \frac{TP}{TP + FN}$$

F1-Score: It is the harmonic mean of precision and recall. It takes both false positive and false negatives into account. Therefore, it performs well on an imbalanced dataset.

$$F1\ Score = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}} = \frac{2(Precision \times Recall)}{2(Precision + Recall)}$$

The dataset is imbalanced (most samples are 'Medium' or 'Good'), so metrics use average='weighted' to account for class distribution. The Confusion Matrix rows/columns correspond to 'Bad' (0), 'Medium' (1), 'Good' (2).

7.2.1 Logistic Regression

Results:

- *Accuracy* = 0.8313

~83.1% of predictions are correct, which is high but inflated by the dominance of 'Good' class predictions (253/320 test samples).

- *Precision* = 0.7819

The weighted precision indicates that ~78.2% of positive predictions are correct, but lower than accuracy due to misclassifications in minority classes.

- *Recall* = 0.8313

The model identifies ~83.1% of true instances, matching accuracy, but skewed toward the majority class.

- *F1 Score* = 0.7819

The F1-score (~79.5%) balances precision and recall, reflecting moderate performance across classes.

- *Confusion Matrix*

	Bad	Medium	High
Bad	0	0	11
Medium	0	13	34
High	0	9	253

The model fails to predict any 'Bad' instances (0/11 correct), predicts only 13/47 'Medium' instances correctly, and excels at 'Good' (253/262). This indicates bias toward the majority class ('Good').

Conclusion:

The performance is good overall but limited by class imbalance. Logistic Regression is a good baseline but

struggles with minority classes, making it less ideal for imbalanced data.

7.2.2 Decision Tree Classifier

Results:

- *Accuracy* = 0.7875

~78.8% of predictions are correct, lower than Logistic Regression, reflecting more balanced predictions across classes.

- *Precision* = 0.8114

~81.1% of positive predictions are correct, higher than Logistic Regression, indicating better precision for predicted classes.

- *Recall* = 0.7875

The model identifies ~78.8% of true instances, matching accuracy.

- *F1 Score* = 0.7978

The F1-score (~79.8%) is comparable to Logistic Regression, balancing precision and recall.

- *Confusion Matrix*

	Bad	Medium	High
Bad	2	0	9
Medium	3	28	16
High	10	30	222

The model predicts 2/11 'Bad' correctly (~18.2%), 28/47 'Medium' (~59.6%), and 222/262 'Good' (~84.7%). It performs better on minority classes than Logistic Regression but still struggles with 'Bad'.

Conclusion:

The performance is moderate. Decision Tree Classifier is less suitable due to overfitting and inconsistent performance across classes.

7.2.3 Random Forest Classifier

Results:

- *Accuracy* = 0.8719

The highest accuracy (~87.2%) indicates excellent overall performance.

- *Precision* = 0.8362

~83.6% of positive predictions are correct, reflecting high reliability.

- $Recall = 0.8719$

The model identifies $\sim 87.2\%$ of true instances, matching accuracy.

- $F1\ Score = 0.8516$

The highest F1-score ($\sim 85.2\%$) shows a strong balance of precision and recall.

- *Confusion Matrix*

	Bad	Medium	High
Bad	0	0	11
Medium	0	27	20
High	0	10	252

The model fails to predict 'Bad' (0/11), predicts 27/47 'Medium' ($\sim 57.4\%$), and 252/262 'Good' ($\sim 96.2\%$). It excels at the majority class but struggles with 'Bad'.

Conclusion:

The performance is excellent, with the highest accuracy and F1-score. The ensemble approach reduces overfitting, improving generalization compared to Decision Tree. However, the failure to predict 'Bad' indicates challenges with severe class imbalance.

Random Forest Classifier is highly suitable due to its strong performance and robustness.

7.2.4 Support Vector Classifier (SVC)

Results:

- $Accuracy = 0.8313$

$\sim 83.1\%$ accuracy matches Logistic Regression, driven by strong 'Good' class predictions.

- $Precision = 0.7813$

$\sim 78.1\%$ of positive predictions are correct, similar to Logistic Regression.

- $Recall = 0.8313$

The model identifies $\sim 83.1\%$ of true instances.

- $F1\ Score = 0.7873$

The F1-score ($\sim 78.7\%$) is slightly lower than others, reflecting weaker minority class performance.

- *Confusion Matrix*

The model fails to predict 'Bad' (0/11), predicts 10/47 'Medium' ($\sim 21.3\%$), and 256/262 'Good' ($\sim 97.7\%$). It performs poorly on minority classes.

	Bad	Medium	High
Bad	0	0	11
Medium	0	10	37
High	0	6	256

Conclusion:

The performance is good but limited by class imbalance, similar to Logistic Regression. The low F1-score and poor 'Bad'/'Medium' predictions indicate that the RBF kernel struggles without tuned hyperparameters or imbalance handling.

SVC is suitable but needs optimization to match Random Forest.

7.2.5 Support Vector Classifier (SVC)

Results:

- $Accuracy = 0.8313$

$\sim 83.1\%$ accuracy is strong but lower than Random Forest.

- $Precision = 0.8089$

$\sim 80.9\%$ precision is higher than Logistic Regression and SVC, indicating reliable positive predictions.

- $Recall = 0.8313$

The model identifies $\sim 83.1\%$ of true instances.

- $F1\ Score = 0.8159$

The F1-score ($\sim 81.6\%$) is strong, second only to Random Forest, balancing precision and recall.

- *Confusion Matrix*

	Bad	Medium	High
Bad	1	0	10
Medium	0	20	27
High	5	12	245

The model predicts 1/11 'Bad' ($\sim 9.1\%$), 20/47 'Medium' ($\sim 42.6\%$), and 245/262 'Good' ($\sim 93.5\%$). It performs better on 'Medium' than Logistic Regression and SVC but struggles with 'Bad'.

Conclusion:

The performance is strong, with a high F1-score and better minority class performance than Logistic Regression and

SVC. However, it is slightly outperformed by Random Forest.

Gradient Boosting is highly suitable, with robust performance across classes.

7.3 Model Comparison

The regression and classification models are compared using their primary metrics (R^2 for regression, Accuracy and F1-Score for classification), considering their ability to handle non-linear relationships and class imbalance (for classification).

7.3.1 Regression Models Comparison

Model	MSE	MAE	R^2
Linear Regression	0.3900	0.5035	0.4032
Decision Tree Regressor	0.6219	0.4656	0.0484
Random Forest Regressor	0.3014	0.4220	0.5389
Support Vector Regressor	0.3516	0.4540	0.4620
Gradient Boosting Regressor	0.3644	0.4853	0.4424

Linear Regression: Moderate performance ($R^2 \sim 0.40$) but limited by linear assumptions.

Decision Tree Regressor: Poor performance ($R^2 \sim 0.05$) due to overfitting, nearly equivalent to a baseline model.

Random Forest Regressor: Best performance ($R^2 \sim 0.54$, lowest MSE ~ 0.30 , MAE ~ 0.42), excelling at capturing non-linear patterns.

Support Vector Regressor: Good performance ($R^2 \sim 0.46$) but limited by default hyperparameters.

Gradient Boosting Regressor: Good performance ($R^2 \sim 0.44$) but slightly outperformed by Random Forest.

Bar Plot:

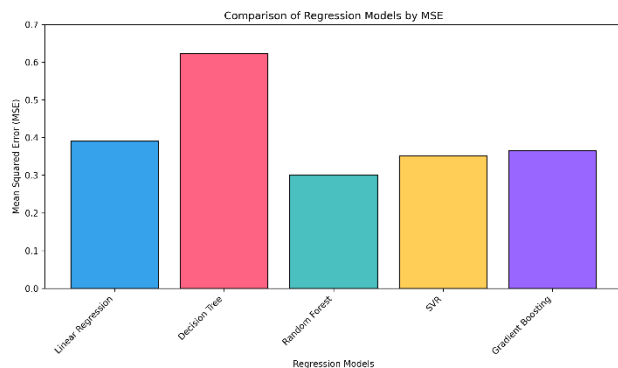


Figure 26. MSE bar plot

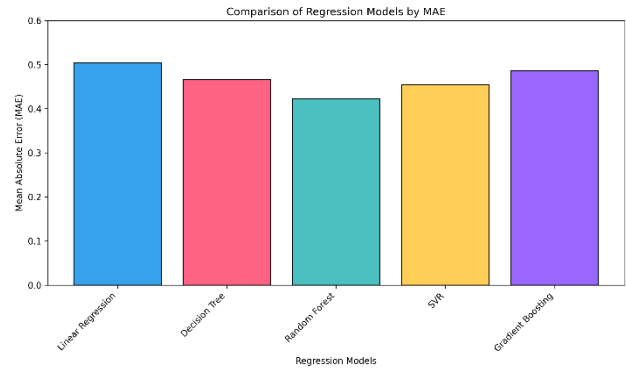


Figure 27. MAE bar plot

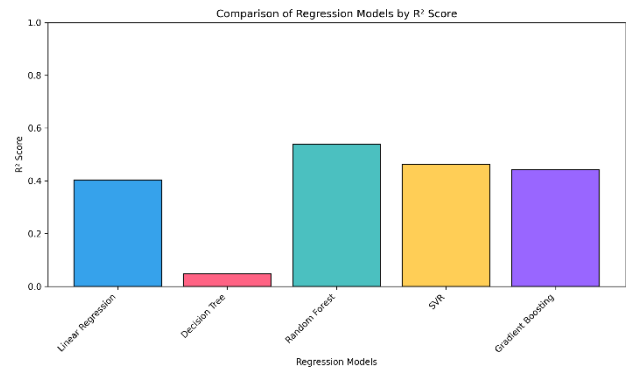


Figure 28. R^2 Score bar plot

7.3.2 Classification Models Comparison

Model	Accuracy	Precision	Recall	F1-Score	Confusion Matrix
Logistic Regression	0.8313	0.7819	0.8313	0.7951	Fails on 'Bad', poor 'Medium'
Decision Tree Classifier	0.7875	0.8114	0.7875	0.7978	Better 'Medium' struggles with 'Bad'
Random Forest Classifier	0.8719	0.8362	0.8719	0.8516	Best overall, fails on 'Bad', strong 'Medium'
Support Vector Classifier	0.8313	0.7813	0.8313	0.7873	Fails on 'Bad', poor on 'Medium'
Gradient Boosting Classifier	0.8313	0.8089	0.8313	0.8159	Good 'Medium', some 'Bad' success

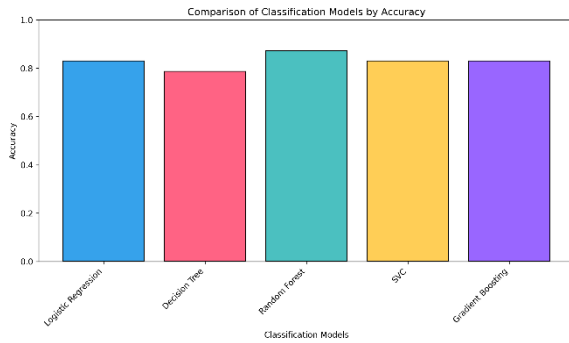
Bar Plot:

Figure 29. Accuracy bar plot

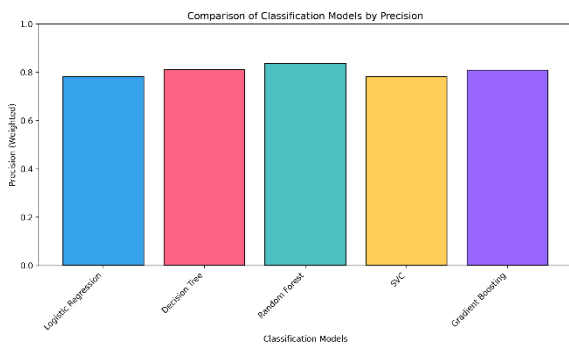


Figure 30. Precision bar plot

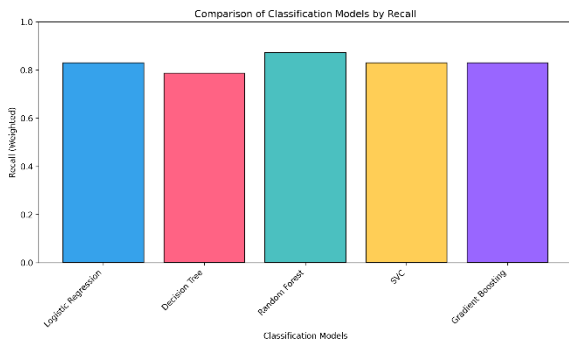


Figure 31. Recall bar plot

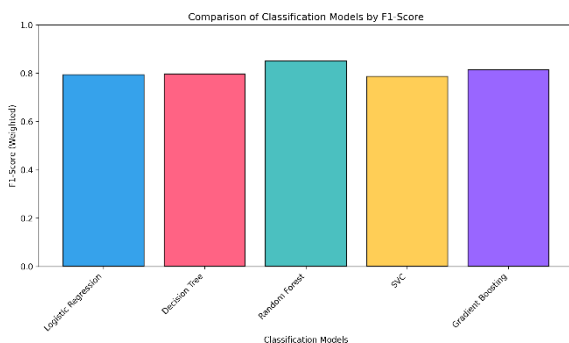


Figure 32. F1-score bar plot

7.4 Conclusion and Best Model Selection

Best Regression Model: Random Forest Regressor. It achieves the lowest MSE (0.3014), lowest MAE (0.4220), and highest R^2 score (0.5389), indicating superior accuracy and explanatory power. Its ensemble approach effectively captures non-linear relationships and feature interactions, making it ideal for predicting numerical quality scores. The R^2 of ~ 0.54 aligns with top-performing models in literature for this dataset.

Best Classification Model: Random Forest Classifier. It achieves the highest Accuracy ($\sim 87.2\%$) and F1-Score ($\sim 85.2\%$), with strong performance on the majority 'Good' class (96.2% correct) and reasonable 'Medium' class performance (57.4%). While it fails to predict the 'Bad' class, this is consistent with the dataset's severe imbalance (few 'Bad' instances). Its robustness and generalization make it the best choice.

8. Conclusion and Recommendations**8.1 Summary of Findings**

The analysis of the red wine quality dataset evaluated five regression models and five classification models to predict wine quality. Regression models predicted numerical quality scores (0–10), while classification models predicted discretized categories. Key findings include:

Regression: Random Forest Regressor outperformed others with the lowest MSE (0.3014), MAE (0.4220), and highest R^2 (0.5389), effectively capturing non-linear relationships. Decision Tree Regressor performed poorly ($R^2=0.0484$), likely due to overfitting.

Classification: Random Forest Classifier achieved the highest Accuracy (0.8719) and F1-Score (0.8516), excelling despite class imbalance. Decision Tree Classifier had the lowest Accuracy (0.7875), while Logistic Regression, SVC, and Gradient Boosting performed comparably (Accuracy=0.8313).

Challenges: The dataset's clustered quality scores (mostly 5–6) limited regression R^2 scores, and severe class imbalance ('Bad' class underrepresented) impacted classification performance, particularly for minority classes.

Recommendation: Classification is better for applications requiring categorical decisions (e.g., quality thresholds), while regression is better for precise score predictions. For this dataset, classification's higher metrics suggest it's more effective, especially if the goal is to categorize wine quality.

1. Can we classify whether wine is good or bad based on quality?

Yes, classification models can categorize wine as 'Bad', 'Medium', or 'Good' quality. However, performance is limited for the 'Bad' class due to imbalance.

2. Can we create a regression model to predict the quality of the given wine?

Yes, regression models can predict numerical quality scores. Accuracy is moderate due to the dataset's limited variance and clustered scores.

8.2 Recommendations for Improvement

To enhance model performance, the following strategies are recommended:

Feature Engineering:

- Create interaction features (e.g., alcohol $\times$ volatile acidity) to capture synergistic effects on quality.
- Apply feature selection (e.g., based on Random Forest feature importance) to reduce multicollinearity (e.g., fixed acidity and pH) and noise (e.g., density).
- Explore additional discretizations for classification (e.g., volatile acidity, sulphates) to simplify patterns for Decision Trees.

Hyperparameter Tuning:

- Optimize parameters for Support Vector Regressor/Classifier (e.g., C, gamma for RBF kernel) using grid search to improve fit.
- Tune Random Forest and Gradient Boosting (e.g., n_estimators, max_depth, learning_rate) to enhance accuracy and reduce overfitting.
- Adjust regularization in Logistic Regression (e.g., C) to handle class imbalance better.

Ensemble and Boosting Methods:

Leverage advanced ensemble techniques like stacking, combining Random Forest and Gradient Boosting predictions to improve robustness.

Apply boosting variants (e.g., XGBoost, LightGBM) for classification to better handle class imbalance.

Use class weighting or oversampling (e.g., SMOTE) in classification to improve minority class ('Low') performance.

These recommendations aim to address the dataset's challenges, enhancing predictive accuracy and robustness for both regression and classification tasks.